

Rings: An Algebraic Structured Peer-to-Peer Network for Decentralized Identity and Resource Ownership

Ryan J. Kung

Abstract

Rings is an implemented structured peer-to-peer network whose design is stated as algebraic carriers, ownership maps, protocol transitions, and explicit proof obligations. The project starts from a concrete failure mode in contemporary Internet systems: application servers, DNS names, relay services, and platform accounts often become the authority that identities and data must trust. Rings instead places identities and resources over a CorrectChord overlay, connects browser and native runtimes through WebRTC/WebAssembly adapters, and keeps cryptographic protocols on carriers separate from the routing identifier space. This paper distinguishes implemented mechanisms from scoped verification evidence and from evaluation obligations that still require Rings-specific measurement. The core object is

$$\text{Rings} = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathsf{T}_A^\mathcal{E}, \Pi, \mathbb{G}, \mathcal{X})$$

with

$$\begin{aligned} R &= \mathbb{Z}/2^{160}\mathbb{Z}, \\ \mathcal{R}_N &= \text{Correct-Chord overlay object}, \\ \Omega_N(X, H) &= (X, \text{owner}_N \circ H), \\ \tau_\nu &: S_\nu \times I_\nu \rightarrow \mathsf{T}_{A_\nu}^{\mathcal{E}}(S_\nu \times O_\nu). \end{aligned}$$

$$\{D, V, M, H, Q, W\} \subset \text{Obj}(\mathbf{Set}/R), \quad \text{CryptoCarrier} \cap R = \emptyset.$$

I. Introduction

Centralized application platforms simplify deployment, but they also concentrate identity, naming, storage, and traffic control at service operators. Cloud and platform centralization therefore becomes a privacy, availability, and autonomy risk for users who need direct communication and durable control over their data [1]–[3]. The Rings project is motivated by this observation: a sovereign network should let a participant own an identity, establish a session, route to another identity or resource, and run application protocols without turning a single application server into the root of authority.

Rings is implemented as a network, but this paper is organized around the algebraic structure that makes the implementation coherent. It separates implemented mechanisms from ideal carrier, ownership, transition, and safety laws that remain obligations for stronger verification and evaluation. The older intuition was that Decentralized Identifiers (DIDs) directly form a finite ring suitable for all routing and cryptographic operations. The refined model is more precise: Chord identifiers form an additive circular carrier for placement, interval order, replica selection, and finger targets; threshold schemes, signatures, encryption, and zero-knowledge protocols use their own fields or groups.

The resulting design has three practical consequences. First, DIDs and virtual DIDs (VIDs) share the same 160-bit placement surface, so identities, mailboxes, service descriptors, chunks, subrings, and application rendezvous points can be routed by one owner function. Second, Rings builds on Chord and CorrectChord rather than replacing the overlay literature: successor lists, predecessor relations, join, rectify, and stabilize are the maintenance substrate on which Rings semantics are layered [4], [5]. Third, state transitions return typed effects instead of directly performing network IO, which allows the overlay to be reasoned about as a pure state machine and then interpreted by browser or native runtimes.

II. Problem Statement and Design Goals

Rings addresses a narrow but recurring gap between existing decentralized systems. Kademlia-derived systems provide scalable lookup, but the XOR metric does not by itself give Rings the circular ownership law needed for successor lists, replicated ownership, and interval-based placement. Relay networks and mixnets improve metadata resistance, but they are not structured P2P ownership systems. Blockchains provide global ordering, but embedding consensus in every message or storage operation would turn Rings into a ledger rather than an algebraic structured peer-to-peer substrate.

The problem is therefore to specify a network object **Rings** that simultaneously supports self-certifying identity and delegated sessions, structured lookup and

resource ownership, browser-native and native transport surfaces, application protocols with typed side effects, cryptographic protocols over correct carriers, and bounded algebraic and operational obligations for routing and state repair.

This paper makes the following contributions. It presents Rings as an algebraic structured peer-to-peer network, not merely as an overlay lookup routine. It models the Rings identifier space as the additive cyclic group $R = \mathbb{Z}/2^{160}\mathbb{Z}$, not as a general finite field. It lifts resource placement into **Set**/ R , so each protocol object carries an explicit map into the overlay. It uses CorrectChord as the maintenance law before introducing Rings-specific replication, storage, mailbox, relay, and ranking semantics. It gives a writer effect monad for DHT transitions and a TLA-style state relation for scoped model checking and trace replay. These algebraic and operational artifacts are the core of the paper; the surrounding text explains which parts are implemented, which parts are checked in bounded models, and which parts remain future measurement obligations.

III. Algebraic System Model

Assumption 1 (System scope).

$$\begin{aligned} \text{fault} &= \text{fail-stop}, \\ \text{auth}(m, \sigma, k) &= V(k, \sigma, m), \\ \text{net} &= \text{async}, \\ \text{io} &= I_X : A_X^* \rightarrow \text{IO}, \\ \text{consensus} &\notin \text{Overlay}, \\ \text{order} &= \text{SidecarWitness}, \\ \text{crypto_ops} \cap \text{route_ops} &= \emptyset. \end{aligned}$$

The fail-stop clause scopes overlay maintenance; identity, ranking, and application protocols still carry adversarial predicates at their own layer.

Definition 1 (Identifier carrier).

$$\begin{aligned} M &= 2^{160}, \\ R &= \mathbb{Z}/M\mathbb{Z}, \\ \delta_a(x) &= (x - a) \bmod M, \\ (a, b]_R &= \{x \in R : 0 < \delta_a(x) \leq \delta_a(b)\}, \\ \text{RouteOps}_R &= \{0, +, -, \delta_a, (_, _)]_R\}, \\ \text{Mult}_R &\notin \text{RouteOps}_R. \end{aligned}$$

Definition 2 (Live topology).

$$\begin{aligned} N &\subset R, \quad N \neq \emptyset, \\ \text{owner}_N(k) &= \arg \min_{u \in N} \delta_k(u), \\ \text{pred}_N(n) &= \arg \min_{p \in N \setminus \{n\}} \delta_p(n), \\ \text{Succ}_N^q(n) &= \text{take}_q(\text{sort}_{\delta_n}(N \setminus \{n\})), \\ \delta_a|_N &= \text{injective}. \end{aligned}$$

Assumption 2 (Correct-Chord stable base).

$$\begin{aligned} r &= |\text{Succ}_N^r(n)|, \\ \text{Steady}(N) &\Rightarrow |N| \geq r + 1, \\ |N| < r + 1 &\Rightarrow \text{Init}(N), \\ \text{Maintain} &= \{\text{join}, \text{rectify}\} \\ &\quad \cup \{\text{stabilize}\}_{\text{CorrectChord}}. \end{aligned}$$

This is the CorrectChord stable base of Zave [5].

Definition 3 (Affine replica set).

$$\begin{aligned} \rho &\in \mathbb{N}_{>0}, \\ \text{repkey}_i^\rho(k) &= k + \lfloor M \cdot i / \rho \rfloor, \\ \text{RepKey}^\rho(k) &= \langle \text{repkey}_i^\rho(k) : 0 \leq i < \rho \rangle, \\ \text{RepOwner}_N^\rho(k) &= \langle \text{owner}_N(\text{repkey}_i^\rho(k)) : 0 \leq i < \rho \rangle. \end{aligned}$$

Storage redundancy is specified first as distinct placement keys. Effective node-level redundancy depends on how those keys map through the live owner function; two placement keys can collapse to the same live owner when the topology is too small or poorly spaced. Successor lists remain the CorrectChord safety root for topology maintenance, not the storage replica set.

Definition 4 (Rings overlay object).

$$\begin{aligned} \mathcal{R}_N &= (R, N, \text{owner}_N, \text{pred}_N, \text{Succ}_N^r, \text{RepOwner}_N^\rho, F, E), \\ F &\subseteq N \times \mathbb{N} \times N, \\ E &= \prod_{v \in \text{VID}} E_v, \\ \text{SafetyRoot}(\mathcal{R}_N) &= (\text{pred}_N, \text{Succ}_N^r), \\ \text{PerfWitness}(\mathcal{R}_N) &= F. \end{aligned}$$

Proposition 1 (Rotation equivariance).

$$\begin{aligned} t \in R, \quad N + t &= \{n + t : n \in N\} \Rightarrow \\ \text{owner}_{N+t}(k + t) &= \text{owner}_N(k) + t. \end{aligned}$$

Proof.

$$\begin{aligned} \forall n \in N. \quad \delta_{k+t}(n + t) &= ((n + t) - (k + t)) \bmod M \\ &= \delta_k(n). \end{aligned}$$

□

Constraint 1 (Carrier separation).

$$\begin{aligned} \text{PlacementOps} &= \{0, +, -, \delta_a, (a, b]_R\} \\ &\quad \cup \{\text{owner}_N\} \\ &\quad \cup \{\text{RepKey}^\rho, \text{RepOwner}_N^\rho\}, \\ \text{CryptoOps} &= \{\times, ^{-1}, \cdot_{\text{scalar}}, \text{interp}\} \\ &\quad \cup \{\text{pair}, \text{subgroup}\}, \\ \text{PlacementOps} \cap \text{CryptoOps} &= \emptyset, \\ \text{CryptoCarrier} &\in \{\mathbb{F}_q, \mathbb{G}, \mathbb{Z}_q\}. \end{aligned}$$

Object	Carrier or law
Chord IDs	additive cyclic group R
DID proof	signature verifier and transcript law
VID	$H_\nu : X_\nu \rightarrow R$ then owner map
placement	
Storage	join semilattice or explicit finalizer
merge	
E2E encryption	group operation plus byte-to-point adapter
Sequencing	partial order over witness domains
Effects	writer monad over action lists

Table I

Carrier boundaries in the Rings model

IV. Architecture As Interfaces

Definition 5 (Layer tuple).

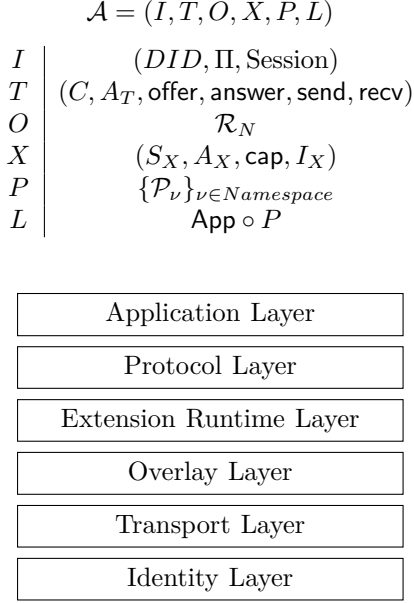


Figure 1. Layers of Rings Network

Definition 6 (Identity proof).

$$\begin{aligned} \Pi &= (K, \Sigma, V, D, \text{enc}, \text{ttl}) \\ V &: K \times \Sigma \times B^* \rightarrow \{\top, \perp\}, \\ D &: K \rightarrow R, \\ \text{enc} &: \Sigma \rightarrow B^*, \\ \text{ttl} &: \Sigma \rightarrow \text{Time}. \end{aligned}$$

Invariant 1 (Session authority).

$$d \in X_D, \quad s \in K.$$

$$\text{Session}(d, s, e) \Rightarrow V(k_d, \sigma_{d \rightarrow s}, d \| s \| e) = \top \wedge \text{now} < e.$$

$$\text{HopCheck}(d, s, e) \equiv \text{Session}(d, s, e).$$

Definition 7 (Transport interface).

$$\begin{aligned} T &= (C, \text{offer}, \text{answer}, \text{send}, \text{recv}) \\ A_T &= \{\text{Offer}, \text{Answer}, \text{Ice}, \text{Open}, \text{Send}, \text{Recv}, \text{Close}\}. \\ I_T : A_T^* &\rightarrow \text{IO}_{WebRTC} \cup \text{IO}_{Native}. \end{aligned}$$

Definition 8 (Runtime interpreter).

$$\begin{aligned} \mathcal{X} &= (S_X, A_X, \text{cap}, I_X) \\ \text{cap} &\subseteq A_X, \\ I_X &: A_X^* \rightarrow \text{IO}, \\ I_X(\epsilon) &= \text{Noop}, \\ I_X(\alpha \cdot \beta) &= I_X(\alpha); I_X(\beta). \end{aligned}$$

Obligation 1 (Runtime confinement).

$$\begin{aligned} \tau &: S \times I \rightarrow \mathcal{E} + (S \times O \times A_X^*), \\ \text{emit}(\tau(s, i)) &\subseteq \text{cap}, \\ a \notin \text{cap} &\Rightarrow I_X(a) = \text{Reject}(a). \end{aligned}$$

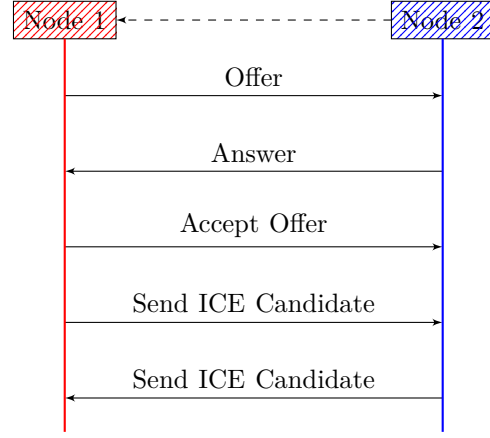


Figure 2. Standard SDP/ICE exchange

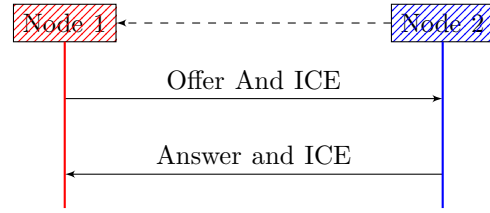


Figure 3. Rings single round-trip SDP/ICE exchange

V. Category and Effects

Definition 9 (Namespace object).

$$\begin{aligned} \nu &\in \{D, V, M, H, U, Q, W\}, \\ H_\nu &: X_\nu \rightarrow R, \\ (X_\nu, H_\nu) &\in \mathbf{Set}/R. \end{aligned}$$

Definition 10 (Placement functor).

$$\begin{aligned} \Omega_N &: \mathbf{Set}/R \rightarrow \mathbf{Set}/N, \\ (X, H) &\mapsto (X, \text{owner}_N \circ H). \\ P_\nu^N &= \text{owner}_N \circ H_\nu : X_\nu \rightarrow N. \end{aligned}$$

Proposition 2 (Placement functoriality).

$$f : X_\nu \rightarrow X_\mu, \quad H_\mu \circ f = H_\nu.$$

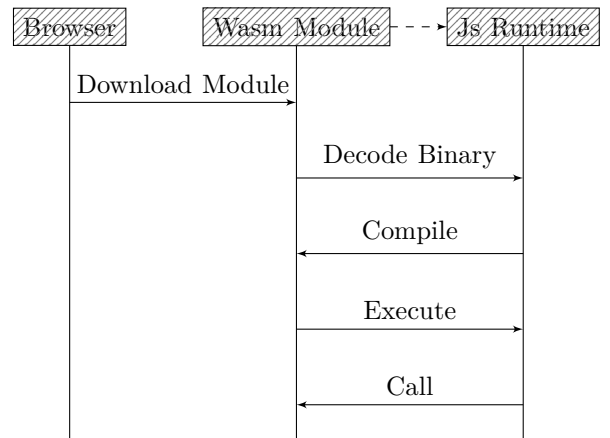


Figure 4. Wasm module execution boundary

$$P_\mu^N \circ f = P_\nu^N.$$

Proof.

$$\text{owner}_N \circ H_\mu \circ f = \text{owner}_N \circ H_\nu.$$

□

Definition 11 (Writer effect monad).

$$\begin{aligned} A^* &= \text{FreeMonoid}(A), \\ \mathsf{T}_A(X) &= X \times A^*, \\ \eta(x) &= (x, \epsilon), \\ (x, \alpha) \gg= f &= \text{let } f(x) = (y, \beta) \text{ in } (y, \alpha \cdot \beta). \end{aligned}$$

Proposition 3 (Effect associativity).

$$(h^* \circ g^*) \circ f^* = h^* \circ (g^* \circ f^*).$$

Proof.

$$((\alpha \cdot \beta) \cdot \gamma) = (\alpha \cdot (\beta \cdot \gamma)).$$

□

Definition 12 (Typed failure stack).

$$\begin{aligned} \mathsf{T}_A^\mathcal{E}(X) &= \mathcal{E} + (X \times A^*). \\ \pi_A(e \in \mathcal{E}) &= \epsilon, \\ \pi_A(x, \alpha) &= \alpha, \\ \text{RecoverableReport} &\in A. \end{aligned}$$

Definition 13 (Protocol object).

$$\begin{aligned} \mathcal{P}_\nu &= (X_\nu, S_\nu, I_\nu, O_\nu, A_\nu, \mathcal{E}_\nu, H_\nu, \tau_\nu) \\ \tau_\nu : S_\nu \times I_\nu &\rightarrow \mathsf{T}_{A_\nu}^\mathcal{E}(S_\nu \times O_\nu). \end{aligned}$$

Definition 14 (Protocol morphism).

$$\begin{aligned} \Sigma : \mathcal{P}_\nu &\rightarrow \mathcal{P}_\mu = (\sigma_X, \sigma_S, \sigma_I, \sigma_O, \sigma_\mathcal{E}, \sigma_A^*) \\ \sigma_A^* &: A_\nu^* \rightarrow A_\mu^*, \\ \sigma_A^*(\alpha \cdot \beta) &= \sigma_A^*(\alpha) \cdot \sigma_A^*(\beta), \\ H_\mu \circ \sigma_X &= H_\nu, \\ \mathsf{T}_{\sigma_A^*}^{\sigma_\mathcal{E}}(\sigma_S \times \sigma_O) \circ \tau_\nu &= \tau_\mu \circ (\sigma_S \times \sigma_I). \end{aligned}$$

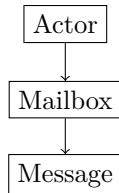


Figure 5. Actor Model

VI. Chord Overlay

$$\begin{aligned} \text{Base} &= \text{Chord}, \\ \text{Maintenance} &= \text{CorrectChord}, \\ \text{Object} &= \mathcal{R}_N, \\ \text{Witnesses} &= \{\text{Algorithms 1, 2, 3}\}. \end{aligned}$$

This section fixes Chord as the base protocol and CorrectChord as the maintenance law [4], [5].

Definition 15 (Local node state).

$$\begin{aligned} S_n &= (p_n, L_n, F_n, E_n) \\ p_n &\in N \cup \{\perp\}, \\ L_n &\in N^{\leq r}, \\ F_n &: \mathbb{N} \rightarrow N, \\ E_n &\subseteq E, \\ p_n &= \text{pred}_N(n), \quad L_n = \text{Succ}_N^r(n). \end{aligned}$$

Definition 16 (Finger target).

$$\begin{aligned} \text{target}_i(n) &= n + 2^i, \\ \text{finger}_i(n) &= \text{owner}_N(\text{target}_i(n)), \\ F_n(i) &\in \{\perp, \text{finger}_i(n)\}. \end{aligned}$$

□

Algorithm 1 DHT Join Transitions

```

1: function BeginJoin( $n, known$ )
2:    $\text{pred}[n] \leftarrow \text{none}$ 
3:    $a \leftarrow \text{FindSuccessor}(n.\text{did})$ 
4:   return  $\text{Remote}(known, a)$ 
5: function InstallSuccessor( $n, s$ )
6:    $\text{succ}[n] \leftarrow \text{take}(r, [s])$ 
7:   return  $\text{Remote}(s, \text{QuerySuccList}(s))$ 
8: function InstallSuccList( $n, s, list$ )
9:    $\text{succ}[n] \leftarrow \text{take}(r, s :: list)$ 
10:  if  $\text{head}(\text{succ}[n]) \neq \text{none}$  then
11:     $h \leftarrow \text{head}(\text{succ}[n])$ 
12:     $\text{notify} \leftarrow \text{Notify}(h, n.\text{did})$ 
13:     $\text{sync} \leftarrow \text{SyncStorage}(h)$ 
14:    return  $\text{Multi}(\text{notify}, \text{sync})$ 
15:  else
16:    return  $\text{Noop}$ 

```

Algorithm 2 DHT Lookup Algorithm

```

1: function Lookup( $key, node$ )
2:    $s \leftarrow \text{head}(\text{succ}[node])$ 
3:   if  $s = \text{none}$  then
4:     return  $\text{Repair}(\text{QuerySuccList}(node))$ 
5:   if  $key \in (node.\text{did}, s]$  then
6:     return  $\text{Local}(s)$ 
7:    $\text{next} \leftarrow \text{closest\_preceding\_node}(node, key)$ 
8:   if  $\text{next} = \text{none}$  then
9:     return  $\text{Repair}(\text{QuerySuccList}(s))$ 
10:  else
11:     $a \leftarrow \text{FindSuccessor}(key)$ 
12:    return  $\text{Remote}(\text{next}, a)$ 

```

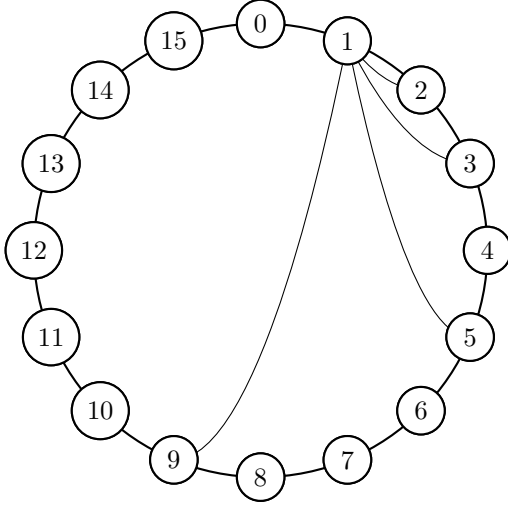


Figure 6. Chord finger targets from node 1 in a 16-node ring

Algorithm 3 DHT Stabilization

```

1: procedure Stabilize( $n, topo$ )
2:    $actions \leftarrow []$ 
3:    $c \leftarrow topo.pred$ 
4:    $s \leftarrow head(succ[n])$ 
5:   if  $c \neq none$  and  $c \in (n.did, s)$  then
6:      $succ[n] \leftarrow insert(succ[n], c)$ 
7:      $actions \leftarrow actions :: FindSuccessor(c)$ 
8:    $succ[n] \leftarrow take(r, merge(succ[n], topo.succ))$ 
9:   if  $head(succ[n]) \neq none$  then
10:     $h \leftarrow head(succ[n])$ 
11:     $actions \leftarrow actions :: Notify(h, n.did)$ 
12:   return  $Multi(actions)$ 

```

Invariant 2 (Ring fixpoint).

$$Inv_{ring}(N, S) \equiv \forall n \in N. p_n = pred_N(n) \wedge L_n = Succ_N^r(n).$$

Invariant 3 (Lookup owner preservation).

$$\begin{aligned} & Steady(N) \wedge Inv_{ring}(N, S) \\ & \Rightarrow Lookup(S, k) \Downarrow owner_N(k). \\ & \Downarrow = Reach_{A_D}(2, 3). \end{aligned}$$

VII. DID and Resource Algebra

Definition 17 (DID namespace).

$$\begin{aligned} \mathcal{D} &= (X_D, H_D, \Pi, Session), \quad H_D : X_D \rightarrow R. \\ D(k) &= H_D(k), \\ owner_N(D(k)) &\in N, \\ Session &\subseteq X_D \times K \times Time. \end{aligned}$$

Definition 18 (VID namespace).

$$\begin{aligned} \mathcal{V} &= (X_V, H_V, Auth_V), \\ H_V &: X_V \rightarrow R, \\ PrivateKey(H_V(x)) &= \perp, \\ Auth_V(op, x, \sigma, caps) &\in \{\top, \perp\}. \end{aligned}$$

Definition 19 (Biased order).

$$A \leq_C B \iff \delta_C(A) \leq \delta_C(B).$$

Algorithm 4 Biased Circular Comparison

```

1: function BiasedCompare( $A, B, C$ )
2:    $a \leftarrow \delta_C(A)$ 
3:    $b \leftarrow \delta_C(B)$ 
4:   if  $a < b$  then
5:     return  $A <_C B$ 
6:   else if  $a > b$  then
7:     return  $B <_C A$ 
8:   else
9:     return  $A = B$ 

```

Definition 20 (Replicated entry algebra).

$$\begin{aligned} \mathcal{E}_v^{join} &= (E_v, \sqsubseteq, \sqcup, \perp), \\ \mathcal{E}_v^{final} &= (E_v, \text{conflict}_v, \text{finalize}_v). \end{aligned}$$

$$\begin{aligned} x \sqcup y &= y \sqcup x, \\ (x \sqcup y) \sqcup z &= x \sqcup (y \sqcup z), \\ x \sqcup x &= x, \\ x \sqcup \perp &= x, \\ \text{finalize}_v &: \text{conflict}_v(E_v) \rightarrow E_v. \end{aligned}$$

case₁ = CRDT.

This is the CRDT join-semilattice case [6].

Invariant 4 (Placement).

$$\begin{aligned} K_{\nu, x} &= \text{replicaKeys}(H_\nu(x)), \\ P_{\nu, x} &= \text{set}(\text{RepKey}^\rho(H_\nu(x))), \\ O_{\nu, x} &= \text{set}(\text{RepOwner}_N^\rho(H_\nu(x))), \\ U_{\nu, x} &= \text{owner}_N[K_{\nu, x}], \\ Inv_{place} &\equiv \forall \nu, x. K_{\nu, x} \subseteq P_{\nu, x} \wedge U_{\nu, x} \subseteq O_{\nu, x}. \end{aligned}$$

Definition 21 (Mailbox).

$$\begin{aligned} mbox(d, e) &= H_M(\text{rings.mailbox} \| d \| e), \\ entry_M &= (\eta, exp, proof_s, dst, cipher). \end{aligned}$$

Definition 22 (Hidden service descriptor).

$$\begin{aligned} h &= (\nu, \rho, Intro, Relay, \\ &\quad Cap, K, exp, rank), \\ VID(h) &= H_H(h), \\ Lookup(h) &= P_H^N(h), \\ Valid(h, \sigma) &= Auth_H(h, \sigma). \end{aligned}$$

Definition 23 (Subring).

$$\begin{aligned} \mathcal{S}_u &= (u, N_u, owner_{N_u}, Succ_{N_u}^{r_u}, E_u) \\ u &= H_U(policy), \\ memberlog_u &\in \text{MergeLog} \cup \text{FinalState}. \end{aligned}$$

Algorithm 5 ElGamal Encryption and Decryption

```

1: Input: Encoded group element  $m_G \in \mathbb{G}$ , private key
    $x \in \mathbb{Z}_q$ , public key  $h = g^x$ , generator  $g$ 
2: Output: Group ciphertext  $(c_1, c_2)$ 
3: procedure Encryption
4:   Choose a random integer  $k \in \mathbb{Z}_q$ 
5:   Compute  $c_1 = g^k$ 
6:   Compute  $c_2 = m_G \cdot h^k$ 
7:   return  $(c_1, c_2)$ 
8: procedure Decryption
9:   Compute  $m_G = c_2 \cdot (c_1)^{-x}$ 
10:  return  $m_G$ 

```

VIII. Cryptographic Carriers and Traffic

Definition 24 (End-to-end carrier).

$$\begin{aligned}
 |\mathbb{G}| &= q, \\
 \text{Scalar}(\mathbb{G}) &= \mathbb{Z}_q, \\
 x &\in \mathbb{Z}_q, \\
 h &= g^x, \\
 \mathbb{G} &\neq R.
 \end{aligned}$$

Algorithm 5 is the group-level carrier operation. Byte payloads must first be encoded into curve points and split into bounded frames. The direct ElGamal body is malleable when detached from its signed transport envelope; Rings therefore treats message signatures, DID ownership of the public key, stream identifiers, sequence numbers, and final-frame checks as part of the security boundary rather than as optional metadata.

Definition 25 (Signed encrypted frame).

$$f = (sid, seq, last, cipher, pk_s, \sigma_s, win).$$

$$\begin{aligned}
 m_f &= sid || seq || last || cipher, \\
 \text{FrameValid}(f) &\Leftrightarrow V(pk_s, \sigma_s, m_f) = \top \\
 &\quad \wedge \text{Session}(D(pk_s), sid, e) \\
 &\quad \wedge \text{SeqOK}(sid, seq, last, win).
 \end{aligned}$$

Definition 26 (Secret sharing placement).

$$\begin{aligned}
 P &\in \mathbb{F}_q[X], \\
 sh_i &= (i, P(i)) \in \mathbb{F}_q^2, \\
 H_S(sh_i) &\in R, \\
 \text{reconstruct}(\{sh_i\}) &= P(0) \quad (\mathbb{F}_q\text{-interp.})
 \end{aligned}$$

This is Shamir reconstruction over the separate field carrier [7].

Definition 27 (Relay frame).

$$\begin{aligned}
 r &= (path, dst, \nu, session, payload, report). \\
 \text{RelayValid}(r) &= \text{SessionValid}(session) \\
 &\quad \wedge dst \in R, \\
 \text{SplitVocabulary} &= \text{MSRP}.
 \end{aligned}$$

The split vocabulary follows MSRP [8].

Algorithm 6 End-to-End Chunking Algorithm

```

1: procedure E2E Chunking
2:   Input: data, chunk size
3:   Output: chunks
4:   chunks  $\leftarrow []$ 
5:   total-chunks  $\leftarrow \text{ceil}(\text{len}(\text{data})/\text{chunk-size})$ 
6:   for i in range(total-chunks) do
7:     chunk  $\leftarrow \text{data}[i*\text{chunk-size} : (i+1)*\text{chunk-size}]$ 
8:     append chunk to chunks
9:   return chunks

```

Invariant 5 (Chunk reassembly).

$$\begin{aligned}
 \text{complete}(m) &\Leftrightarrow \forall i < \text{total}(m). \exists! c_i \\
 &\quad \wedge \sum_i |c_i| \leq B_m \\
 &\quad \wedge \text{now} < \text{ttl}(m).
 \end{aligned}$$

IX. Ordering, Ranking, and Security

Definition 28 (Sidecar witness).

$$\begin{aligned}
 w &= (\text{domain}, \text{root}, \text{height}, \text{time}, \text{finality}). \\
 c &= H(\text{msg}), \\
 \text{Witness}(\text{msg}, w) &\Leftrightarrow \text{Include}(c, w.\text{root}) = \top \\
 &\quad \wedge \text{Final}(w) = \top.
 \end{aligned}$$

Definition 29 (Witness order).

$$\begin{aligned}
 a < b &\Leftrightarrow \text{domain}(w_a) = \text{domain}(w_b) \\
 &\quad \wedge \text{time}(w_a) < \text{time}(w_b). \\
 \chi &: \text{Domain}_a \times \text{Domain}_b \rightarrow \text{Order}.
 \end{aligned}$$

Definition 30 (Ranking event).

$$\begin{aligned}
 q &= (\text{subject}, \text{rater}, \text{behavior}, \\
 &\quad \text{evidence}, \text{epoch}, \text{weight}, \sigma). \\
 m_q &= \text{subject} || \text{behavior} || \text{evidence} || \text{epoch}, \\
 \text{RankValid}(q) &\Leftrightarrow V(k_{\text{rater}}, \sigma, m_q) = \top.
 \end{aligned}$$

Definition 31 (Score aggregation).

$$\begin{aligned}
 \text{score}_e(s) &= \text{robust}\{\text{weight}(q) \cdot \text{value}(q) : \\
 &\quad q.\text{subject} = s\}.
 \end{aligned}$$

$$\text{robust} \in \{\text{median}, \text{trimmedMean}, \text{declared}\}.$$

Attack	Predicate	Required boundary
Sybil	many DIDs, one actor	admission and ranking policy
Replay	stale valid σ	TTL, stream order, final marker
MITM	wrong key for DID	DID proof and signed envelope
DoS	unbounded resource use	quotas and chunk bounds

Table II

Security predicates and required controls

Table II is a boundary summary, not a standalone proof of Sybil resistance, anonymity, or denial-of-service tolerance. Those properties depend on protocol-specific admission, ranking, relay, and resource-accounting policies outside the base overlay.

Obligation 2 (Replay rejection).

$$\begin{aligned} \text{accept}(s, \eta, t) &\Rightarrow \eta \notin \text{Seen}_s \wedge t < \text{exp}(s), \\ \text{Seen}'_s &= \text{Seen}_s \cup \{\eta\}, \\ \text{emitDelivery} &< \text{Seen}'_s. \end{aligned}$$

X. Verification Evidence and Evaluation Scope

This section separates three things that are easy to conflate: algebraic specification, scoped verification evidence, and performance evaluation. The repository contains executable evidence for specific overlay, storage, and message-flow obligations. It does not yet contain a wide-area Rings benchmark that can replace a published Chord latency or churn experiment.

Definition 32 (State relation).

$$\begin{aligned} C &= (S, Q) \\ \frac{Q = a :: Q_0 \quad I_A(a) = i \quad \tau(S, i) = (S', O, \beta)}{(S, Q) \rightarrow (S', Q_0 \cdot \beta)} \\ \text{Explore}_{\text{scope}} &= \text{ModelCheck}(\rightarrow, \Gamma) \end{aligned}$$

The exploration relation follows TLA+ style and Statertight execution for a finite scope Γ [9], [10]. The scope must state the node count, topology shape, message scheduler, and failure model; otherwise the result is only an implementation test, not a model-checking claim.

Invariant 6 (Effect refinement).

$$\begin{aligned} \text{Impl}_A(s, i) &\preceq I_A^\sharp(\tau(s, i)). \\ \pi_{\text{state}}(\text{Impl}_A) &= \pi_{\text{state}}(I_A^\sharp \circ \tau). \end{aligned}$$

Obligation 3 (Topology coverage).

$$\begin{aligned} \mathcal{L} &= \{\text{line}, \text{wrap}, \text{gap}, \text{partition}, \text{merge}, \text{churn}\} \\ \mathcal{S} &= \{\text{loss}, \text{dup}, \text{reorder}, \text{timeout}, \text{restart}\}. \\ \text{CheckCase} &= (L, S, P), \quad L \in \mathcal{L}, \quad S \in \mathcal{S}. \end{aligned}$$

Artifact	Checked scope	Limitation
Topology tests	join, rectify, stabilize fixpoints on finite DID layouts	not a parametric TLAPS proof
Statertight	predecessor update, small discovery abstraction, storage CRDT, handoff	scoped models, not a full live network
Trace replay	deterministic delivery schedules over real routing code	not an Internet latency benchmark
E2E tests	signed direct-ElGamal frame order, final marker, and replay behavior	not a symbolic cryptographic proof

Table III

Verification evidence and explicit limits

Definition 33 (Latency decomposition).

$$\begin{aligned} T_{\text{total}} &= T_{\text{conn}} + h \cdot T_{\text{hop}} + T_{\text{crypto}} + T_{\text{app}} + T_{\text{witness}}. \\ h &= O(\log |N|) \quad (F_n \text{ populated}). \end{aligned}$$

Definition 34 (Reliability baseline).

$$\begin{aligned} \text{failrate} &= \frac{|F|}{|Q|}, \\ \text{timeoutMean} &= \frac{1}{|Q|} \sum_{q \in Q} \text{to}(q). \end{aligned}$$

Obligation 4 (Benchmark report).

$$\begin{aligned} \text{Report}_{\text{Rings}} &= (N, \rho, r, \text{transport}, \text{runtime}, \\ &\quad \text{scheduler}, T_{\text{total}}, \text{timeoutMean}, \text{failrate}). \\ \text{Baseline}_{\text{Chord}} &\neq \text{Measurement}_{\text{Rings}}. \end{aligned}$$

Chord's published hop-count and churn measurements remain the baseline for the underlying circular overlay [4]. A Rings performance claim must add the measured cost of connection establishment, browser or native transport, cryptographic framing, storage replication, and application witnesses on a specified implementation build.

XI. Related Work

Distributed hash tables give Rings its closest technical ancestry. IPFS and libp2p use Kademlia-derived routing, where distance is measured by XOR and routing state is organized into buckets [11]–[13]. This is a strong fit for content-addressed lookup, but Rings needs a different law: the owner of a DID or VID is the first live node clockwise from that point, topology repair follows successor-list maintenance, and storage redundancy places affine rotations of an entry key around the ring. The paper therefore uses Chord's circular order as the base model, then uses Zave's CorrectChord results to avoid treating the original Chord maintenance protocol as already correct under churn [4], [5].

Relay anonymity systems take a different route. Tor and Nym route traffic over selected relay paths and focus on hiding communication metadata from local observers or intermediate services [14], [15]. Rings does not replace these systems as anonymity systems. It instead specifies a structured ownership substrate on which relay, mailbox, and hidden-service style protocols can be written. This distinction matters: a relay path can carry traffic, but it does not define the algebraic owner of a DID, mailbox, chunk, or service descriptor.

Blockchain and account-based decentralized systems provide another partial answer. A ledger can publish names, order events, and make writes globally auditable, but the price is global consensus on operations that often only need local routing, replicated storage, or protocol-specific ordering. Rings keeps global consensus outside the base overlay. When a protocol needs stronger ordering evidence, it uses sidecar witnesses rather than changing the overlay into a blockchain.

Browser-native networking is also part of the design space. WebRTC gives browsers a standardized peer transport, while WebAssembly gives portable code a browser execution target [16], [17]. Rings treats these as adapters for the same abstract protocol object, not as separate networks. The same DID/session, Chord owner, effect, and resource-placement laws should hold for browser nodes and native nodes; only the IO interpreter changes.

System	Metric	Ownership law
Kademlia	XOR	bucket routing
Relay	path	no structured owner
mixnets		
Rings	circular	clockwise owner, successor list, affine placement keys

Table IV
Routing and ownership comparison [11]–[15]

XII. Conclusion

Rings is an algebraic structured peer-to-peer network: structured because ownership, routing, affine storage replication, and repair are defined over CorrectChord’s circular overlay, and algebraic because each layer preserves an explicit carrier or morphism rather than hiding semantics inside ad hoc network procedures.

$$\text{Rings} = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathbf{T}_A^\mathcal{E}, \Pi, \mathbb{G}, \mathcal{X}).$$

$$\begin{aligned} \text{Overlay} &= \text{CorrectChord}(R, N, r), \\ \text{Resource} &= (X_\nu, H_\nu) \in \mathbf{Set}/R, \\ \text{Placement} &= \Omega_N(X_\nu, H_\nu), \\ \text{Transition} &= \tau_\nu : S_\nu \times I_\nu \rightarrow Y_\nu, \\ Y_\nu &= \mathbf{T}_{A_\nu}^\mathcal{E}(S_\nu \times O_\nu), \\ \text{Crypto} &= \mathbb{G} \vee \mathbb{F}_q, \quad \text{Crypto} \cap R = \emptyset, \\ \text{Evidence} &= \{\text{scopedModels}, \text{traceReplay}\} \\ &\quad \cup \{\text{invariants}, \text{benchmarkObligations}\}. \end{aligned}$$

References

- [1] N. Kshetri, “Privacy and security issues in cloud computing: The role of institutions and institutional evolution,” *Telecommunications Policy*, vol. 37, no. 4–5, pp. 372–386, 2013.
- [2] D. Zissis and D. Lekkas, “Addressing cloud computing security issues,” *Future Generation Computer Systems*, vol. 28, no. 3, pp. 583–592, 2012.
- [3] N. Kshetri and S. Murugesan, “Cloud computing and EU data privacy regulations,” *Computer*, vol. 46, no. 3, pp. 86–89, 2013.
- [4] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, “Chord: A scalable peer-to-peer lookup service for internet applications,” in *Proceedings of the 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*. ACM, 2001, pp. 149–160.
- [5] P. Zave, “Reasoning about identifier spaces: How to make Chord correct,” *IEEE Transactions on Software Engineering*, vol. 43, no. 12, pp. 1144–1156, 2017, <https://arxiv.org/abs/1610.01140>.
- [6] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Stabilization, Safety, and Security of Distributed Systems*, ser. Lecture Notes in Computer Science, vol. 6976. Springer, 2011, pp. 386–400.
- [7] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [8] B. Campbell, R. Mahy, and C. Jennings, “The message session relay protocol (MSRP),” RFC 4975, 2007, <https://www.rfc-editor.org/info/rfc4975>.

- [9] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [10] J. Nadal, “Stateright: Model checking for implementing distributed systems,” <https://www.stateright.rs/>, accessed: July 4, 2026.
- [11] P. Maymounkov and D. Mazières, “Kademlia: A peer-to-peer information system based on the XOR metric,” in *Peer-to-Peer Systems*, ser. Lecture Notes in Computer Science, vol. 2429. Springer, 2002, pp. 53–65.
- [12] J. Benet, “IPFS - content addressed, versioned, P2P file system,” arXiv:1407.3561 [cs.NI], 2014, <https://arxiv.org/abs/1407.3561>.
- [13] Protocol Labs, “libp2p: A modular network stack,” 2026, <https://libp2p.io/>.
- [14] R. Dingledine, N. Mathewson, and P. Syverson, “Tor: The Second-Generation onion router,” in *13th USENIX Security Symposium (USENIX Security 04)*. San Diego, CA: USENIX Association, Aug. 2004, <https://www.usenix.org/conference/13th-usenix-security-symposium/tor-second-generation-onion-router>.
- [15] C. Diaz, H. Halpin, and A. Kiayias, “The Nym network: The next generation of privacy infrastructure,” Whitepaper, 2021, <https://nym.com/nym-whitepaper.pdf>.
- [16] H. Alvestrand, “Overview: Real-time protocols for browser-based applications,” RFC 8825, 2021, <https://www.rfc-editor.org/rfc/rfc8825>.
- [17] WebAssembly Working Group, “WebAssembly core specification,” W3C Recommendation, 2019, <https://www.w3.org/TR/2019/REC-wasm-core-1-20191205/>.